

Week 13: Introduction to Node.js & Express

1. Node.js Runtime & Event-Driven Architecture

Node.js is a runtime environment that allows JavaScript to run outside the browser. It uses an event-driven, non-blocking I/O architecture which makes it lightweight and efficient for backend development. This helps applications handle multiple requests at the same time without slowing down performance. Node.js is commonly used for scalable web applications and real-time systems.

2. Modules & npm Packages

Modules in Node.js are used to divide code into reusable and manageable files. Built-in modules like `http`, `fs`, and `path` provide useful functionalities for development. `npm` (Node Package Manager) is used to install external libraries and dependencies required in projects. Using `npm` packages reduces development time and improves efficiency.

3. Basic Express Server Setup

Express.js is a lightweight and flexible Node.js framework used for creating web servers and REST APIs. A basic Express server was created to understand how requests are handled between client and server. It provides features like routing, middleware, and simplified server-side coding. Express makes backend development faster and more organized.

4. Routing in Express

Routing is used to define how the server responds to client requests such as GET, POST, PUT, and DELETE. Different routes can be created for handling different pages or API endpoints. This helps in building structured backend applications and improves maintainability.

5. Serving Static Files

Express can serve static files such as HTML, CSS, JavaScript, images, and other frontend assets. Static middleware is used to make these files publicly accessible. This is useful for connecting frontend interfaces with backend services and hosting web-based content.

6. Middleware in Express

Middleware functions are used to process requests before sending responses. They can be used for logging, authentication, validation, and error handling. Middleware improves application security and request management.

7. API Testing & Server Execution

Node.js applications are run using commands like `node app.js` or `npm start`. APIs created in Express can be tested using browsers, Postman, or frontend integration. This helps verify if routes and server responses are working correctly.

Conclusion

This week focused on backend development fundamentals using Node.js and Express.js. By understanding runtime architecture, modules, npm packages, routing, middleware, static file handling, and server setup, a strong backend foundation was built. These concepts are essential for developing scalable and efficient MERN stack applications.